

Verilog 模板大全

此文档将常用 Verilog 模块以分层清晰的 Markdown 形式呈现，便于查阅与复用。各模块包含中文/英文名称说明与对应代码块。

目录

1. 一位全加器 (1-bit Full Adder)
2. 多位全加器 (Multi-bit Adder, 行为级)
3. 4 位串行进位加法器 (Ripple Carry Adder)
4. D 触发器 (带异步复位)
5. JK 触发器 (带异步复位)
6. T 触发器 (带异步复位)
7. 3-8 译码器
8. 十进制优先编码器 (10-4)
9. 二分频器
10. 偶数分频器 (通用 N 分频)
11. 奇数分频器 (50% 占空比)
12. 4 选 1 多路复用器 (Mux)
13. 1 分 4 数据分配器 (Demux)
14. 三态缓冲器 (Tri-state Buffer)
15. 七段显示译码器 (0-F)
16. 奇校验位发生器
17. 偶校验位发生器

- 18. 计数器: 同步置数, 异步清零
- 19. 计数器: 同步置数, 同步清零
- 20. 计数器: 异步置数, 异步清零
- 21. 计数器: 异步置数, 同步清零
- 22. 莫尔型状态机 (Moore)
- 23. 米莱型状态机 (Mealy)

1. 一位全加器 (1-bit Full Adder)

- 功能: $\text{sum} = a \oplus b \oplus \text{cin}$, $\text{cout} = (a \& b) \mid (a \& \text{cin}) \mid (b \& \text{cin})$

```
module full_adder_1bit (  
    input wire a,  
    input wire b,  
    input wire cin, // 进位输入  
    output wire sum, // 和  
    output wire cout // 进位输出  
);  
// 逻辑描述:  $\text{sum} = a \oplus b \oplus \text{cin}$ ;  $\text{cout} = ab + ac + bc$   
assign sum = a ^ b ^ cin;  
assign cout = (a & b) | (a & cin) | (b & cin);  
  
// 也可以直接用:  $\text{assign \{cout, sum\} = a + b + cin}$ ;  
endmodule
```

2. 多位全加器 (Multi-bit Adder) - 行为级描述

- 参数: WIDTH (默认 4)
- 说明: 行为级最简洁, 综合器会自动优化结构

```

module full_adder_multibit #(
    parameter WIDTH = 4
)(
    input wire [WIDTH-1:0] a,
    input wire [WIDTH-1:0] b,
    input wire      cin,
    output wire [WIDTH-1:0] sum,
    output wire      cout
);
// 行为级描述最简洁，综合器会自动优化结构
assign {cout, sum} = a + b + cin;
endmodule

```

3. 由一位全加器例化组成的多位全加器 (4-bit Ripple Carry Adder)

- 结构: 串行进位，例化 4 个一位全加器

```

module ripple_carry_adder_4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    input wire      cin,
    output wire [3:0] sum,
    output wire      cout
);
    wire c0, c1, c2; // 中间进位信号

    // 例化一位全加器
    // 格式: 模块名 实例名 (端口连接);
    full_adder_1bit u0 (
        .a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c0)
    );

    full_adder_1bit u1 (

```

```
.a(a[1]), .b(b[1]), .cin(c0), .sum(sum[1]), .cout(c1)
);

full_adder_1bit u2 (
.a(a[2]), .b(b[2]), .cin(c1), .sum(sum[2]), .cout(c2)
);

full_adder_1bit u3 (
.a(a[3]), .b(b[3]), .cin(c2), .sum(sum[3]), .cout(cout)
);
endmodule
```

4. D 触发器 (D Flip-Flop) - 带异步复位

```
module d_ff (
input wire clk,
input wire rst_n, // 低电平复位
input wire d,
output reg q
);
always @(posedge clk or negedge rst_n) begin
if (!rst_n)
q <= 1'b0;
else
q <= d;
end
endmodule
```

5. JK 触发器 (JK Flip-Flop)

```
module jk_ff (
input wire clk,
input wire rst_n,
```

```
input wire j,  
input wire k,  
output reg q  
);  
always @(posedge clk or negedge rst_n) begin  
if (!rst_n) begin  
q <= 1'b0;  
end  
else begin  
case ({j, k})  
2'b00: q <= q;    // 保持  
2'b01: q <= 1'b0; // 置0  
2'b10: q <= 1'b1; // 置1  
2'b11: q <= ~q;   // 翻转  
endcase  
end  
end  
endmodule
```

6. T 触发器 (T Flip-Flop)

```

module t_ff (
    input wire clk,
    input wire rst_n,
    input wire t,    // Toggle enable
    output reg q
);
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 1'b0;
    else if (t)
        q <= ~q; // T为1时翻转
    else
        q <= q; // T为0时保持
    end
endmodule

```

7. 3-8 译码器 (3-to-8 Decoder)

```

module decoder_3_8 (
    input wire [2:0] in,
    output reg [7:0] out
);
always @(*) begin
    case (in)
        3'b000: out = 8'b00000001;
        3'b001: out = 8'b00000010;
        3'b010: out = 8'b00000100;
        3'b011: out = 8'b00001000;
        3'b100: out = 8'b00010000;
        3'b101: out = 8'b00100000;
        3'b110: out = 8'b01000000;
        3'b111: out = 8'b10000000;
        default: out = 8'b00000000;
    endcase
end
endmodule

```

8. 十进制优先编码器 (10线-4线)

- 说明: 输入 `in[9:0]` 代表十进制数 0-9, 输出为 BCD 码; `in[9]` 优先级最高。

```
module priority_encoder_10_4 (  
    input wire [9:0] in,  
    output reg [3:0] out,  
    output reg      valid // 指示是否有输入被激活  
);  
always @(*) begin  
    valid = 1'b1;  
    if    (in[9]) out = 4'd9;  
    else if (in[8]) out = 4'd8;  
    else if (in[7]) out = 4'd7;  
    else if (in[6]) out = 4'd6;  
    else if (in[5]) out = 4'd5;  
    else if (in[4]) out = 4'd4;  
    else if (in[3]) out = 4'd3;  
    else if (in[2]) out = 4'd2;  
    else if (in[1]) out = 4'd1;  
    else if (in[0]) out = 4'd0;  
    else begin  
        out = 4'd0;  
        valid = 1'b0; // 无输入激活  
    end  
end  
endmodule
```

9. 二分频器 (Divide-by-2)

```
module clk_div_2 (  
    input wire clk_in,
```

```

input wire rst_n,
output reg clk_out
);
always @(posedge clk_in or negedge rst_n) begin
if (!rst_n)
clk_out <= 1'b0;
else
clk_out <= ~clk_out; // 每个上升沿翻转一次，实现除以2
end
endmodule

```

10. 偶数分频器 (通用 N 分频)

- 参数: DIV_N (默认 6)
- 说明: 计数到 $DIV_N/2 - 1$ 时翻转

```

module clk_div_even #(
parameter DIV_N = 6
)(
input wire clk_in,
input wire rst_n,
output reg clk_out
);
// 计数器位宽计算 (自动计算需要多少位)
reg [31:0] cnt;

// 偶数分频逻辑: 计数到 N/2 - 1 时翻转
always @(posedge clk_in or negedge rst_n) begin
if (!rst_n) begin
cnt <= 0;
clk_out <= 1'b0;
end
else begin
if (cnt == (DIV_N / 2) - 1) begin
cnt <= 0;

```



```
clk_out <= ~clk_out;
end
else begin
cnt <= cnt + 1;
end
end
end
endmodule
```

11. 通用奇数分频器 (50% 占空比)

- 参数: `DIV_N` (默认 5, 必须为奇数)
- 原理: 上/下降沿分别产生脉冲并取交集以得 50% 占空比

```
module clk_div_odd #(
    parameter DIV_N = 5
)(
    input wire clk_in,
    input wire rst_n,
    output wire clk_out
);
reg [31:0] cnt;
reg    clk_p; // 上升沿触发的时钟
reg    clk_n; // 下降沿触发的时钟

// 上升沿计数与翻转
always @(posedge clk_in or negedge rst_n) begin
    if (!rst_n) begin
        cnt <= 0;
        clk_p <= 0;
    end
    else begin
        if (cnt == DIV_N - 1)
            cnt <= 0;
        else
```

```

cnt <= cnt + 1;

// 产生占空比非50%的波形, 高电平持续 (N-1)/2 + 1 个周期
// 例如 N=5, (5-1)/2 = 2. cnt为 0,1,2 时高 (3个周期), 3,4 时低
if (cnt <= (DIV_N - 1) / 2)
    clk_p <= 1'b1;
else
    clk_p <= 1'b0;
end
end

// 下降沿采样 clk_p 得到 clk_n (相当于延迟半个周期)
always @(negedge clk_in or negedge rst_n) begin
    if (!rst_n)
        clk_n <= 0;
    else
        clk_n <= clk_p;
    end

// 输出为两者的与, 从而获得 50% 占空比
// 原理: P宽(N+1)/2, N延迟0.5. 重叠部分为 (N+1)/2 - 0.5 = N/2
assign clk_out = clk_p & clk_n;
endmodule

```

12. 数据选择器 (4选1 Mux)

```

module mux_4to1 (
    input wire [3:0] in,
    input wire [1:0] sel,
    output reg      out
);
always @(*) begin
    case (sel)
        2'b00: out = in[0];
        2'b01: out = in[1];
        2'b10: out = in[2];

```

```
2'b11: out = in[3];
endcase
end
endmodule
```

13. 数据分配器 (1分4 Demux)

```
module demux_1to4 (
    input wire    in,
    input wire [1:0] sel,
    output reg [3:0] out
);
always @(*) begin
    out = 4'b0000; // 默认全0
    case (sel)
        2'b00: out[0] = in;
        2'b01: out[1] = in;
        2'b10: out[2] = in;
        2'b11: out[3] = in;
    endcase
end
endmodule
```

14. 三态门 (Tri-state Buffer)

```
module tri_state_buffer (
    input wire in,
    input wire en, // 高电平使能
    output wire out
);
assign out = (en) ? in : 1'bz;
endmodule
```

15. 七段显示译码器 (0-F)

- 假设: 共阴极, 段序 `abcdefg`

```
module decoder_hex_7seg (  
    input wire [3:0] hex_in,  
    output reg [6:0] seg_out // {a, b, c, d, e, f, g}  
);  
    always @(*) begin  
        case (hex_in)  
            //      abcdefg  
            4'h0: seg_out = 7'b1111110;  
            4'h1: seg_out = 7'b0110000;  
            4'h2: seg_out = 7'b1101101;  
            4'h3: seg_out = 7'b1111001;  
            4'h4: seg_out = 7'b0110011;  
            4'h5: seg_out = 7'b1011011;  
            4'h6: seg_out = 7'b1011111;  
            4'h7: seg_out = 7'b1110000;  
            4'h8: seg_out = 7'b1111111;  
            4'h9: seg_out = 7'b1111011;  
            4'hA: seg_out = 7'b1110111;  
            4'hB: seg_out = 7'b0011111;  
            4'hC: seg_out = 7'b1001110;  
            4'hD: seg_out = 7'b0111101;  
            4'hE: seg_out = 7'b1001111;  
            4'hF: seg_out = 7'b1000111;  
            default: seg_out = 7'b0000000;  
        endcase  
    end  
endmodule
```

16. 奇校验位发生器 (Odd Parity Generator)

```
module parity_gen_odd #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] data_in,
    output wire          parity_bit
);
    assign parity_bit = ~(^data_in);
endmodule
```

17. 偶校验位发生器 (Even Parity Generator)

```
module parity_gen_even #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] data_in,
    output wire          parity_bit
);
    assign parity_bit = ^data_in;
endmodule
```

18. 计数器：同步置数，异步清零

```
module cnt_sync_load_async_clr #(
    parameter MOD = 10,
    parameter WIDTH = 4
)(
    input wire          clk,
    input wire          rst_n, // 异步清零 (低电平)
    input wire          load_en, // 同步置数
    input wire [WIDTH-1:0] load_val,
    output reg [WIDTH-1:0] cnt
);
    always @(posedge clk or negedge rst_n) begin
```

```
if (!rst_n)
cnt <= 0;
else if (load_en)
cnt <= load_val;
else if (cnt == MOD - 1)
cnt <= 0;
else
cnt <= cnt + 1;
end
endmodule
```

19. 计数器：同步置数，同步清零

```
module cnt_sync_load_sync_clr #(
parameter MOD = 10,
parameter WIDTH = 4
)(
input wire      clk,
input wire      rst_n, // 同步清零 (低电平)
input wire      load_en, // 同步置数
input wire [WIDTH-1:0] load_val,
output reg [WIDTH-1:0] cnt
);
always @(posedge clk) begin
if (!rst_n)
cnt <= 0;
else if (load_en)
cnt <= load_val;
else if (cnt == MOD - 1)
cnt <= 0;
else
cnt <= cnt + 1;
end
endmodule
```

20. 计数器：异步置数，异步清零

- 注意: 通常异步复位优先级高于异步置数，或由单元库定义

```
module cnt_async_load_async_clr #(
    parameter MOD = 10,
    parameter WIDTH = 4
)(
    input wire      clk,
    input wire      rst_n, // 异步清零
    input wire      load_en, // 异步置数 (假设高电平有效)
    input wire [WIDTH-1:0] load_val,
    output reg [WIDTH-1:0] cnt
);
// 敏感列表中包含 load_en
always @(posedge clk or negedge rst_n or posedge load_en) begin
    if (!rst_n)
        cnt <= 0;
    else if (load_en)
        cnt <= load_val;
    else if (cnt == MOD - 1)
        cnt <= 0;
    else
        cnt <= cnt + 1;
    end
endmodule
```

21. 计数器：异步置数，同步清零

```
module cnt_async_load_sync_clr #(
    parameter MOD = 10,
    parameter WIDTH = 4
)(
```

```

input wire      clk,
input wire      rst_n, // 同步清零
input wire      load_en, // 异步置数
input wire [WIDTH-1:0] load_val,
output reg [WIDTH-1:0] cnt
);
// 异步信号在敏感列表中, 同步信号在 always 块内部判断
always @(posedge clk or posedge load_en) begin
    if (load_en) begin
        cnt <= load_val;
    end
    else begin
        // 此时为时钟上升沿
        if (!rst_n)
            cnt <= 0;
        else if (cnt == MOD - 1)
            cnt <= 0;
        else
            cnt <= cnt + 1;
        end
    end
end
endmodule

```

22. 莫尔型 (Moore) 状态机

- 特点: 输出只与当前状态有关
- 示例: 序列检测 (检测 11)

```

module fsm_moore #(
    parameter S0 = 2'b00,
    parameter S1 = 2'b01,
    parameter S2 = 2'b10
)(
    input wire clk,
    input wire rst_n,

```



```

input wire in,
output reg out
);
reg [1:0] current_state, next_state;

// 1. 状态跳转逻辑 (时序逻辑)
always @(posedge clk or negedge rst_n) begin
if (!rst_n)
current_state <= S0;
else
current_state <= next_state;
end

// 2. 下一状态判断逻辑 (组合逻辑)
always @(*) begin
case (current_state)
S0: next_state = (in) ? S1 : S0;
S1: next_state = (in) ? S2 : S0;
S2: next_state = (in) ? S2 : S0;
default: next_state = S0;
endcase
end

// 3. 输出逻辑 (组合逻辑, 只与状态有关)
always @(*) begin
if (current_state == S2)
out = 1'b1;
else
out = 1'b0;
end
endmodule

```

23. 米莱型 (Mealy) 状态机

- 特点: 输出与当前状态和输入都有关
- 示例: 序列检测 (检测 11)

```

module fsm_mealy #(
    parameter S0 = 1'b0,
    parameter S1 = 1'b1
)(
    input wire clk,
    input wire rst_n,
    input wire in,
    output reg out
);
    reg current_state, next_state;

    // 1. 状态跳转
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            current_state <= S0;
        else
            current_state <= next_state;
        end

    // 2. 下一状态判断
    always @(*) begin
        case (current_state)
            S0: next_state = (in) ? S1 : S0;
            S1: next_state = (in) ? S1 : S0; // 检测到1, 如果是1继续保持S1(重叠检测)
            default: next_state = S0;
        endcase
    end

    // 3. 输出逻辑 (与状态 和 输入 都有关)
    always @(*) begin
        case (current_state)
            S0: out = 1'b0;
            S1: out = (in) ? 1'b1 : 1'b0; // 在S1状态下, 如果输入为1, 立即输出1 (检测到11)
            default: out = 1'b0;
        endcase
    end
endmodule

```

以上内容整理自原始 Verilog 模板，统一以 Markdown 呈现，层次分明，便于检索与复制。